

TP 3 : récursivité

ZFAI

1 Fonctions récursives : exercices d'introduction

On présente une façon d'écrire les fonctions en informatique : les fonctions récursives. De façon informelle, il s'agit de fonctions qui s'écrivent ainsi :

- il existe des cas de bases pour lesquels on connaît la valeur ;
- en supposant que l'on connaît la valeur pour certains cas, on arrive à construire les valeurs pour les cas suivants.

Remarque. Pour simplifier, une fonction récursive est une fonction qui s'appelle elle-même. La définition d'une fonction récursive est très proche de la façon dont on définit une suite par récurrence.

Exemple 1. On considère la suite définie par : $u_0 = 3, \forall n \in \mathbb{N}, u_{n+1} = 3u_n - 5$. De manière récursive et en Python, elle est définie ainsi :

```
def u(n) :  
    if n == 0 :  
        return 3  
    return 3*u(n-1) - 5
```

Informatiquement, si on considère le calcul de $u(3)$ les étapes sont les suivantes :

- on cherche $u(3)$ qui est égal à $3*u(2) - 5$
- on cherche alors $u(2)$ qui est égal à $3*u(1) - 5$
- on cherche alors $u(1)$ qui est égal à $3*u(0) - 5$
- On cherche alors $u(0)$ qui est égal à 3.
- On a alors $u(1)$ égal à 4,
- On a alors $u(2)$ égal à 7,
- on a alors $u(3)$ égal à 16

Il est possible pour l'ordinateur de mettre en pause certains calculs qui nécessitent des résultats encore non obtenus. Une fois ces résultats calculés, les premiers calculs peuvent reprendre leur cours.

Exercice 1 (Étude de la programmation de la suite de Fibonacci). On rappelle que la suite de Fibonacci $(F_n)_{n \in \mathbb{N}}$ est définie par : $F_0 = 1, F_1 = 1, \forall n \in \mathbb{N}, F_{n+2} = F_{n+1} + F_n$.

1. (Python) Écrire le code Python suivant :

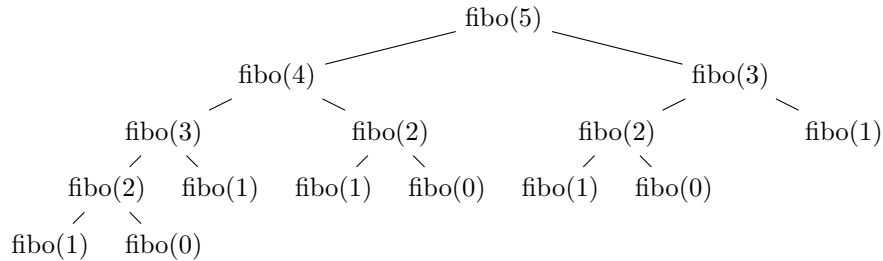
```
def fibo_rec(n) :  
    if n == 0 or n == 1 :  
        return 1  
    return fibo_rec(n-1) + fibo_rec(n-2)
```

2. (Python) Exécuter `fibo_rec(37)`. Que constatez-vous ?

La lenteur s'explique par le nombre de calculs effectués. Pour justifier cela, on cherche à compter le nombre de fois qu'on appelle la fonction `fibo_rec`. Pour tout $n \in \mathbb{N}$, on note C_n le nombre de fois qu'on appelle la fonction `fibo_rec` dans le calcul de `fibo_rec(n)`. Pour $n = 0$ ou $n = 1$, on appelle la fonction 1 fois. Pour tout $n \geq 2$, on a directement la relation de récurrence :

$$C_n = 1 + C_{n-1} + C_{n-2}.$$

3. (Math) En posant la suite auxiliaire $(D_n)_{n \in \mathbb{N}}$ définie par $\forall n \in \mathbb{N}, D_n = C_n + 1$, donner une formule de C_n en fonction de n .
4. En admettant qu'un PC moderne grand public effectue de l'ordre de 10^9 calculs élémentaires en une seconde, estimer le temps que mettrait le PC pour calculer C_{42} .
5. On conclut que cette façon de calculer récursivement n'est pas du tout efficace. Cela est dû au fait qu'il y a une explosion du nombre de calculs. Par exemple, si on cherche à calculer $\text{fibonacci}(5)$, on obtient l'arbre de calcul suivant :



On pourra remarquer que de nombreux calculs sont effectués plusieurs fois. Pour éviter ce problème, une manière de procéder est de garder en mémoire les différents calculs. On peut faire cela avec un dictionnaire par exemple,

```

def fibo(n) :
    dico = {0:1,1:1} # initialisation correspondant à la suite.
    def fibo_aux(n) :
        if n in dico :
            return dico[n]
        V = fibo_aux(n-1) + fibo_aux(n-2)
        dico[n] = V # ajout du nouveau calcul dans le dictionnaire
        return V
    return fibo_aux(n)
  
```

Expliquer qu'avec cette nouvelle version, le calcul de $\text{fibonacci}(n)$ est en $O(n)$ opérations élémentaires.

Correction

1. On recopie simplement le code.
2. On constate que le calcul met quelques secondes, ce qui est lent.
3. (D_n) est une suite récurrente vérifiant :

$$D_0 = 2, D_1 = 2, \forall n \in \mathbb{N}, D_{n+2} = D_{n+1} + D_n.$$

On reconnaît alors une suite récurrente linéaire d'ordre 2. Ainsi après résolution usuelle :

$$\forall n \in \mathbb{N}, D_n = \frac{1 + \sqrt{5}}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n + \frac{\sqrt{5} - 1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n.$$

Donc

$$\forall n \in \mathbb{N}, C_n = \frac{1 + \sqrt{5}}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n + \frac{\sqrt{5} - 1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n - 1.$$

4. En calculant, on trouve que $D_{42} \approx 8 \cdot 10^8$. Donc le temps serait de l'ordre 0,8 secondes. Pourtant, lorsqu'on exécute `fibonacci(42)`, on constate que le temps mis est beaucoup plus long. Cela est dû au fait que les appels de fonctions coûtent plus que des opérations élémentaires et que l'on a pas pris en compte les problèmes de gestion de la mémoire. Bien que l'on ait sous-estimé le temps mis, on pourra remarquer que le calcul de D_{100} avec une machine qui effectuerait 10^{12} calculs par seconde mettrait de l'ordre de 20 ans, ce qui est donc impraticable.
5. Avec cette nouvelle version, dès qu'un nouveau terme est calculé, il est stocké en mémoire, ce qui explique qu'après avoir calculé `fibonacci(n-1)`, on effectue qu'un seul appel dans `fibonacci(n-2)`.

Ainsi, la récurrence est la suivante :

$$\forall n \in \mathbb{N}, C_{n+1} = 2 + C_{n-1}.$$

Ainsi, on retrouve que $C_n = O(n)$.

Exercice 2 (Génération des mots binaires de taille n). Une application de la récursivité est qu'elle permet de générer des structures combinatoires dont on connaît une définition inductive. Pour rappel, un mot binaire de taille $n \geq 1$ est un élément de $\{0, 1\}^n$. On peut remarquer que l'on a :

$$\forall n \geq 1, \{0, 1\}^{n+1} = \{0, 1\} \times \{0, 1\}^n$$

En Python, on choisit de représenter un mot binaire par une chaîne de caractères de 0 et 1. Par exemple, les listes suivantes

```
["0", "1"], ["00", "01", "10", "11"]
```

représentent respectivement $\{0, 1\}$ et $\{0, 1\}^2$.

1. (Python) Tester en Python "1"+"0". Quel est le résultat obtenu ?
2. (Python) À l'aide de l'opération + sur les chaînes de caractères, écrire une fonction récursive `liste_mots_binaires(n)` prenant en argument un entier $n \geq 1$ et qui renvoie la liste des mots binaires de taille n .

Correction

1. On obtient

```
"10"
```

2. On obtient :

```
def liste_mots_binaires(n) :
    if n == 0 :
        return [""]
    else :
        L = liste_mots_binaires(n-1)
        M = []
        for x in L :
            M.append("0"+x)
            M.append("1"+x)
        return M
```

2 Exercices à faire

Exercice 3 (Exponentiation rapide). Étant donné un nombre réel x et un entier naturel n , l'exponentiation rapide est une manière de calculer x^n minimisant le nombre de multiplications. Elle est basée sur la propriété suivante :

$$x^0 = 1, \forall n \in \mathbb{N}, x^n = \begin{cases} (x^2)^{\frac{n}{2}} & \text{si } n \text{ pair,} \\ x \cdot (x^2)^{\frac{n-1}{2}} & \text{sinon} \end{cases}$$

Cette méthode est appelée exponentiation rapide.

1. Écrire une fonction récursive `exponentiation_rapide(x,n)` prenant un réel x et un entier naturel n , et qui renvoie x^n calculé à l'aide de l'exponentiation rapide.
2. Donner un ordre de grandeur du nombre de multiplications effectuées dans une exponentiation rapide.

Correction

1. On obtient comme code Python :

```
def exponentiation(x,n) :
    if n == 0 :
        return 1
    if n%2 == 0 :
        return exponentiation(x**2, n//2)
    else :
        return x*exponentiation(x**2, n//2)
```

2. L'ordre de grandeur du nombre de multiplication est $O(\log_2(n))$.

Exercice 4 (Suite récurrente linéaire). Soient a, b, c trois réels. On considère la suite $(u_n)_{n \in \mathbb{N}}$ définie par :

$$u_0 = a, u_1 = b, u_2 = c, \forall n \in \mathbb{N}, u_{n+3} = 2u_{n+2} - u_{n+1} + 4u_n.$$

1. Écrire une fonction récursive `u(a,b,c,n)` prenant en paramètres trois réels a, b, c et un entier $n \geq 0$ et qui renvoie u_n .
2. Calculer `u(2,0,3, 2000)`.

Correction

1. On peut par exemple écrire :

```
def u(a,b,c,n) :  
    dico = {0:a,1:b,2:c}  
    def u_aux(n) :  
        if n in dico :  
            return dico[n]  
        nouveau_u = 2*u_aux(n-1) - u_aux(n-2) + 4*u_aux(n-3)  
        dico[n] = nouveau_u  
        return dico[n]  
    return u_aux(n)
```

2. Cette question est présente pour vérifier que la fonction récursive que vous avez écrite n'est pas naïve. On trouve :

$$7372349510157 \dots 2925419773.$$

Exercice 5 (Nombre de Catalan). On définit la suite $(c_n)_{n \in \mathbb{N}}$ par

$$c_0 = 1, \forall n \in \mathbb{N}, c_{n+1} = \sum_{k=0}^n c_k c_{n-k}.$$

1. Écrire une fonction récursive naïve `cat(n)` prenant en argument un entier n et qui renvoie c_n .
2. Quel type de problème rencontre-t-on si on essaie de calculer c_{42} ?
3. Écrire une fonction récursive efficace `cat_eff(n)` qui prend en argument un entier n et qui renvoie c_n .
4. Calculer avec la nouvelle fonction c_{42} .

Correction

1. Voici une version naïve :

```
def cat(n) :  
    if n == 0 :  
        return 1  
    S = 0  
    for i in range(n) :  
        S += cat(i)*cat(n-1-i)  
    return S
```

2. On effectue beaucoup d'appels récurifs, et certains calculs identiques sont effectués beaucoup de fois. Ainsi, le calcul final est très lent.
3. Voici une possibilité :

```
def cat_eff(n) :
    dico = {0 : 1}
    def aux(n) :
        if n in dico :
            return dico[n]
        S = 0
        for i in range(n) :
            S += aux(i)*aux(n-1-i)
        dico[n] = S
        return S
    return aux(n)
```

4. Avec cette nouvelle version, on peut facilement calculer c_{42} , qui est égal à : 39044429911904443959240.

Exercice 6 (Triangle de Pascal). Écrire une fonction récursive `ligne_pascal(n)` prenant en argument un entier naturel n et qui renvoie la liste $\left[\binom{n}{k}, 0 \leq k \leq n\right]$. Par exemple, pour $n = 2$, la valeur de retour est $[1, 2, 1]$. On veillera à ne pas avoir une explosion dans les calculs.

Correction

Voici une possibilité :

```
def triangle(n) :
    if n == 0 :
        return [1]
    T = triangle(n-1)
    L = [1]+[0]*(n-1)+[1]
    for i in range(1,n) :
        L[i] = T[i-1]+T[i]
    return L
```

3 Pour les plus rapides

Exercice 7. Il est possible d'utiliser la récursivité pour construire des objets, par exemple l'ensemble des permutations d'une taille donnée. Pour simplifier, on cherche à construire la liste des permutations de $\{0, 1, \dots, n-1\}$. Pour cela, on donne la règle de construction :

- Il existe une unique permutation de taille 1 qui est $(0,)$. L'ensemble correspondant est donc $[(0,)]$.
- Pour construire l'ensemble des permutations de taille n , en supposant qu'on connaisse toutes les permutations de taille $n-1$: on procède ainsi. Pour chaque permutation de taille $n-1$, on insère la valeur $n-1$ aux n positions possibles et chaque insertion nous donne une nouvelle permutation de taille n .

Dans la suite, on représente une permutation de $\{0, 1, \dots, n-1\}$ par un tuple de taille n des éléments de $\{0, 1, \dots, n-1\}$.

Pour rappel, un tuple à un élément s'écrit comme ceci en Python : $(a,)$. Il est possible comme pour les listes de choisir des tranches de tuples et l'opération $+$ permet de concaténer deux tuples.

1. Écrire une fonction `liste_nouveaux(t,a)` qui prend en argument un tuple t , un élément a et qui renvoie une liste de tuples obtenue en insérant a à toutes les positions possibles dans t . Par exemple, `liste_nouveaux((1,0,2),3)` renvoie :

```
[(3,1,0,2),(1,3,0,2),(1,0,3,2),(1,0,2,3)]
```

2. En déduire une fonction `generer_permut(n)` qui prend en argument un entier $n \geq 1$ et qui renvoie toutes la liste de toutes les permutations de $\{0, 1, \dots, n-1\}$.

Correction

1. On a par exemple :

```
def liste_nouveaux(t,a) :
    L = []
    n = len(t)
    for i in range(n+1) :
        L.append(t[:i]+(a,)+t[i:])
    return L
```

2. On peut par exemple écrire :

```
def generer_permut(n) :
    if n == 1 :
        return [(0,)]
    Avant = generer_permut(n-1)
    Tout = []
    for t in Avant :
        L = liste_nouveaux(t,n-1)
        for x in L :
            Tout.append(x)
    return Tout
```

Exercice 8 (Partition d'un entier). On dit qu'un k -uplet $(a_0, \dots, a_{k-1})$ d'entiers strictement positif est une partition de l'entier n si : la séquence est croissante et $\sum_{i=0}^{k-1} a_i = n$. Compléter le code suivant qui permet de générer toutes les partitions de l'entier n .

```
def generer_part(n) :
    dico = {1:[(1,)], 2 : [(1,1),(2,)]}
    def generer_aux(n) :
        if n in dico :
            return dico[n]
        . . .
    return dico[n]
```

Correction

Voici une façon de compléter ce code :

```
def partition(n) :
    dico = {1:[(1,)],2:[(1,1),(2,)]}
    def partition_aux(n) :
        if n in dico :
            return dico[n]
        L = []
        L.append((n,))
        for k in range(n-1,0,-1) :
            d = partition_aux(n-k)
            for x in d :
                if x[-1] <= k :
                    L.append(x+(k,))
        dico[n] = L
    return L
return partition_aux(n)
```